Sec4DevOps Services

CI/CD Pipeline report

TM and © 2020 - 2024, All Rights Reserved

TABLE OF CONTENTS

1. Introduction....3
2. CI/CD Design....3
3. Applied Security Checks....6
   3.1 Secrets Management....6
   3.2 SAST....6
   3.3 DAST....7
4. Implementation....7
   4.1 CI pipeline....7
   4.2 CD pipeline....13
5. Threat Analysis....14
6. Notes....14
7. Summary....15
8. Version History....15

TM and © 2020 - 2024, All Rights Reserved

1. Introduction

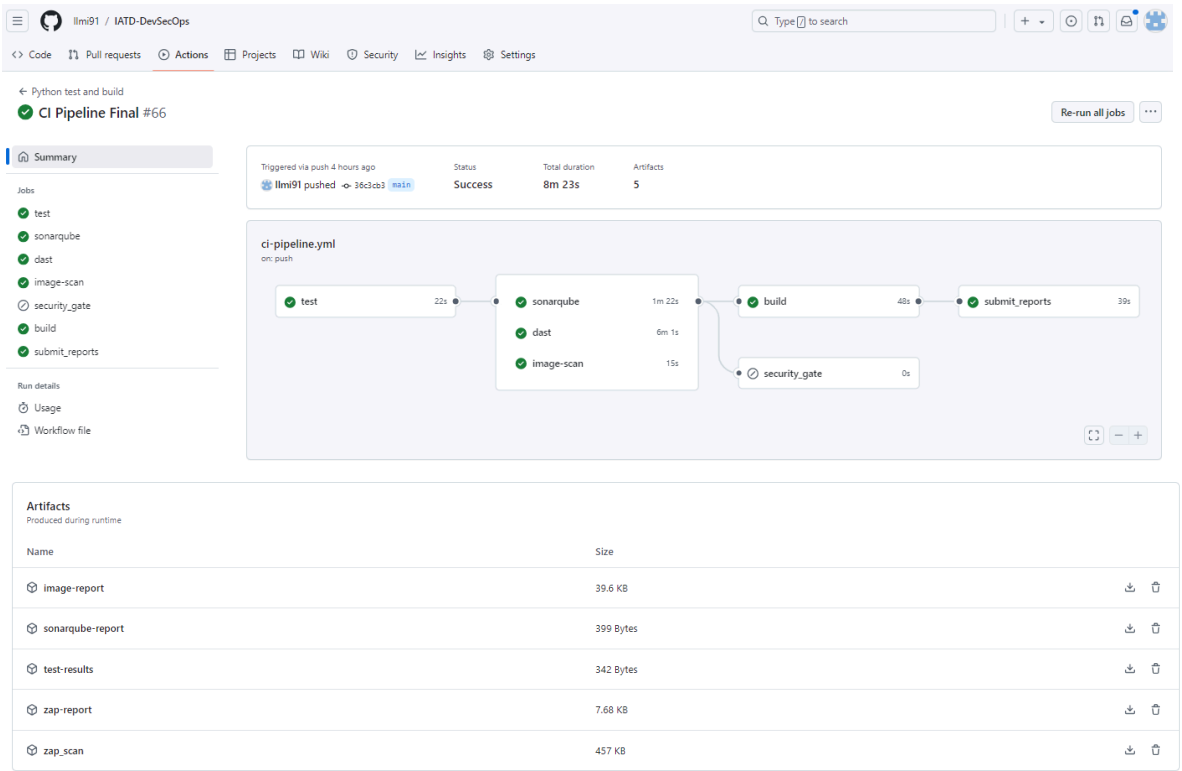
The purpose of this report is to document the implemented CI/CD pipeline for a secure web application. This document describes a technical high-level solution of security implementation to ensure the safe environment for the web application.

2. CI/CD Design

A pipeline in the context of DevOps is a set of automated processes and tools that allows both developers and operators work together in managing the process of building, testing, and deploying software. Pipelines are the core component of Continuous Integration (CI) and Continuous Deployment/Delivery. CI/CD pipelines ensures that every code changes are tested and validated before being deployed to production, this enhancing code reliability, quality efficiency.

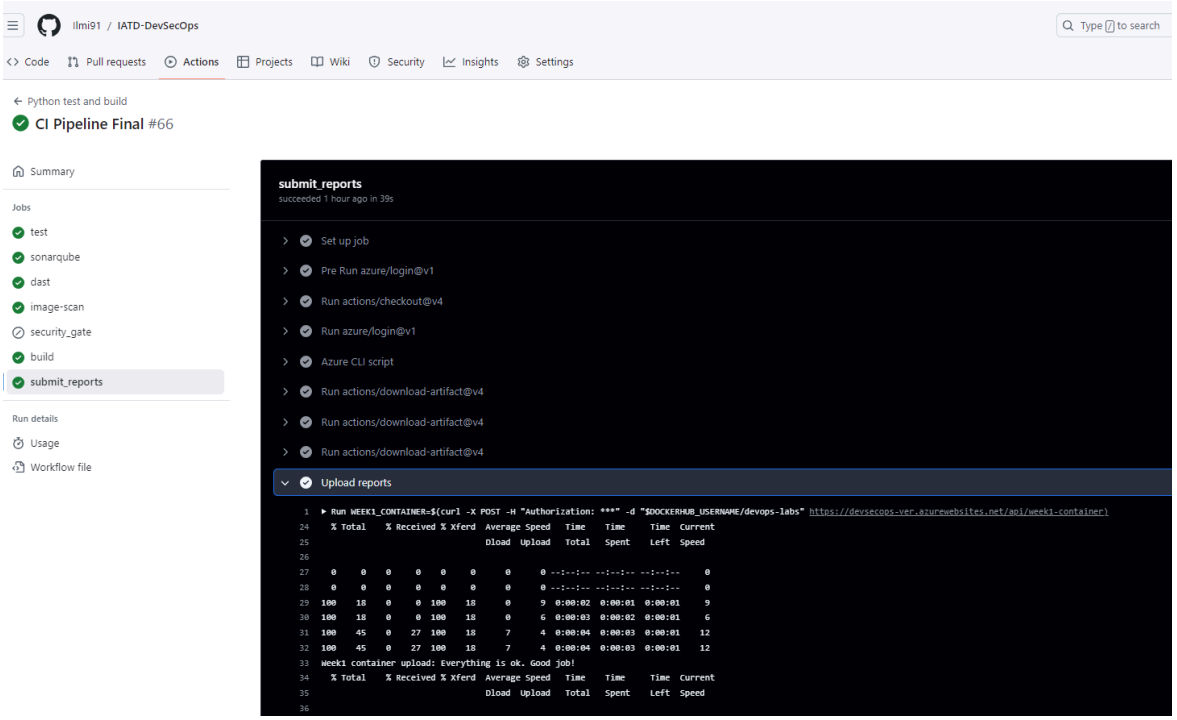
To design a CI/CD pipeline for a secure web application, we utilized GitHub as a source code repository, Docker Hub for container management, VS Code as an IDE for code development, and Azure Cloud to deploy our application. The pipeline runs in the GitHub repository. By linking our GitHub repository directly with Docker Hub and Azure ensuring seamless integration and deployment for our application.

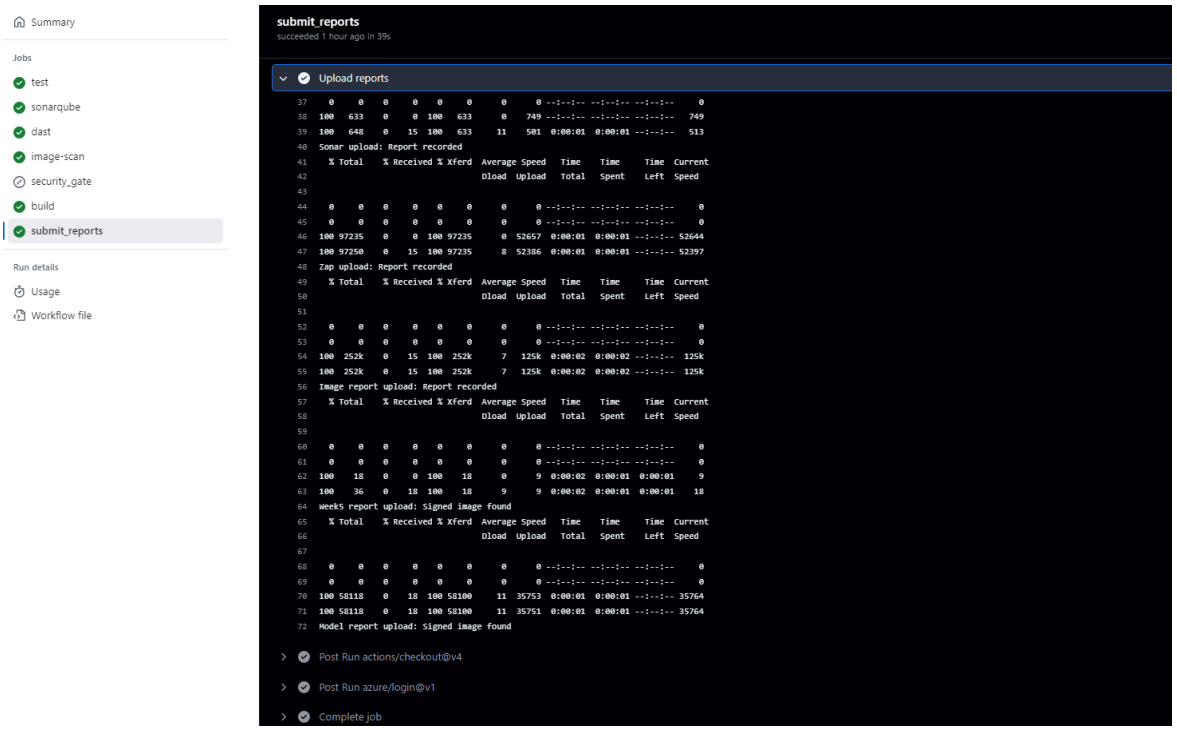
The integration can be seen in the image below which shows the **CI pipeline** diagram that successfully ran in GitHub repository, and artifacts were generated from the pipeline.



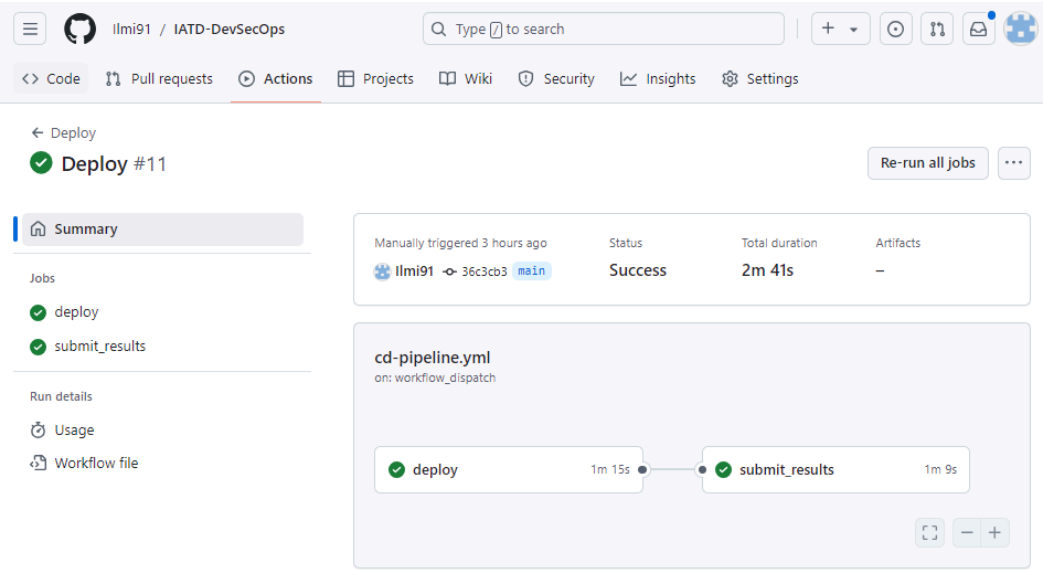
TM and © 2020 - 2024, All Rights Reserved

This images below show the reports that successfully uploaded based on the CRTC lab activities week 1, 3, 4, 5 and 6. Which include container upload report, SonarQube report, ZAP report, signed image report, and threat model report.

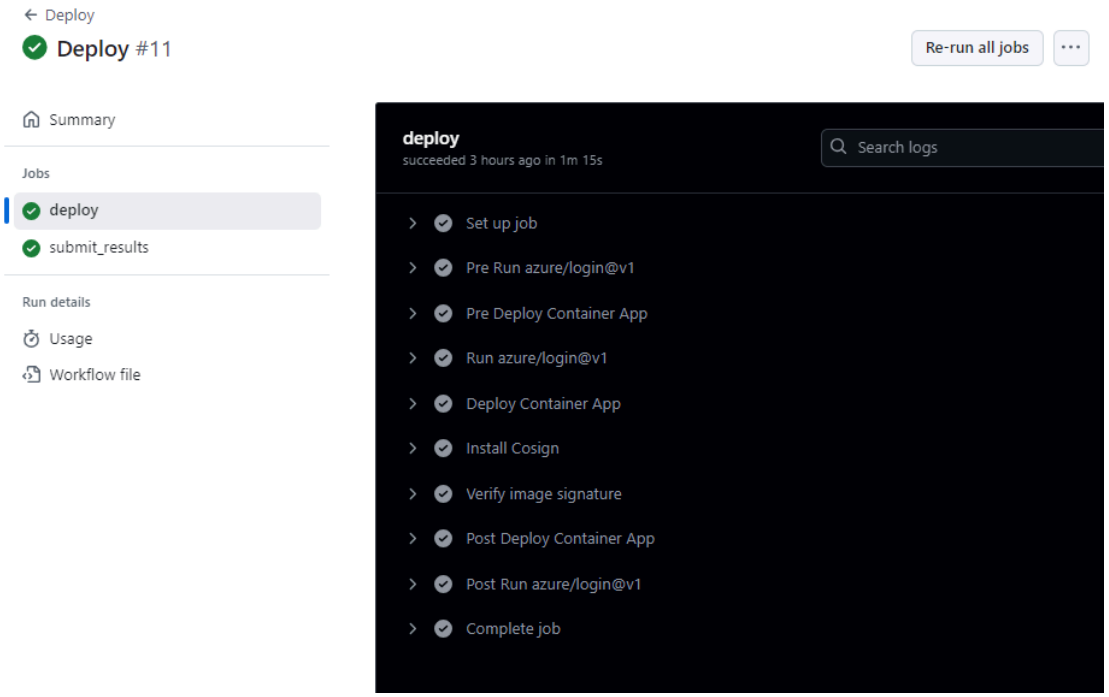




Continuous Delivery (CD) pipeline automates the process of application from development to production. This image below shows the **CD pipeline** diagram successfully ran in GitHub repository.

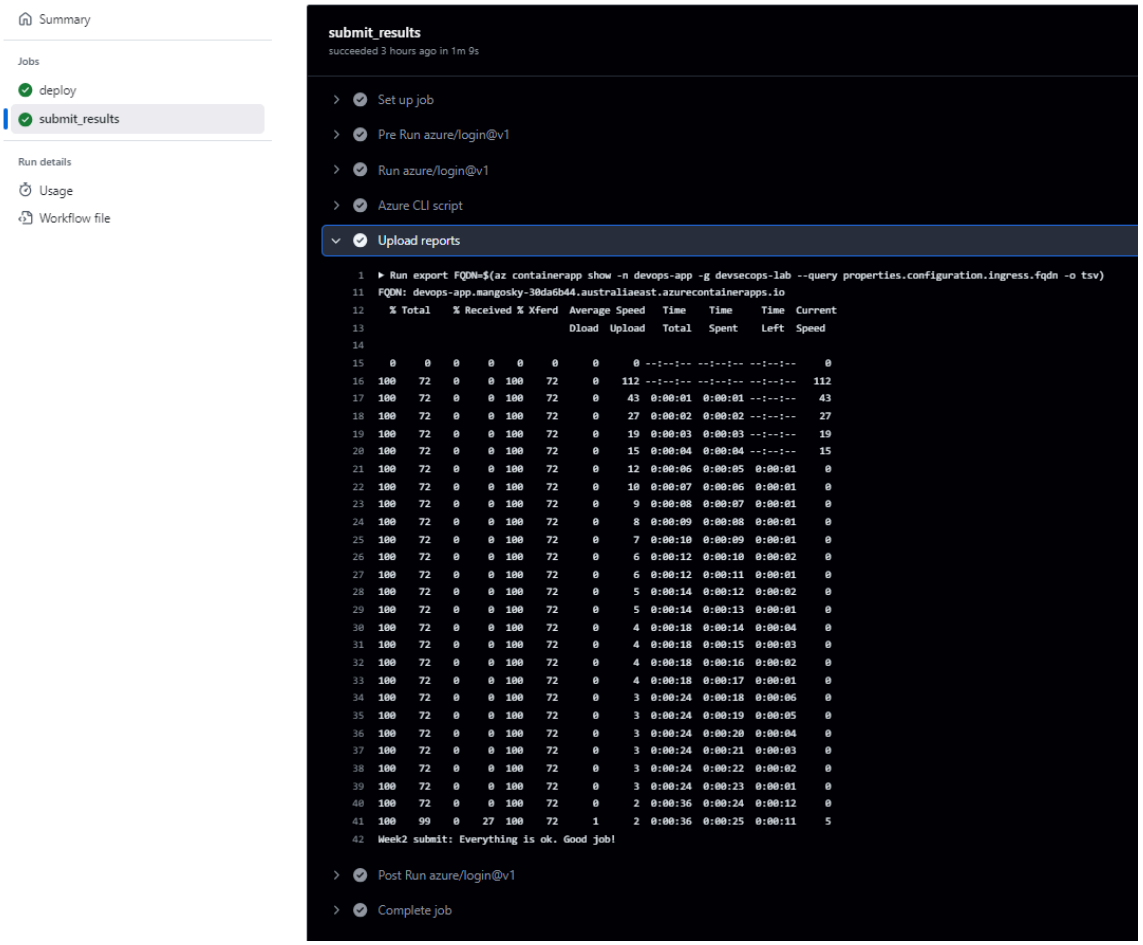


Deploy in CD pipeline is a crucial step that automates the process of deploying application to various environments. Below is the image shows the deploy job in our CD pipeline.



Submit results step is the process of collecting and reporting the outcomes of various stages in the pipeline. From the image below we can see that the report has been uploaded based on the week 2 CRTC lab activities.

TM and © 2020 - 2024, All Rights Reserved



3. Applied Security Checks

To secure our application we applied security checks by setting up Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools in our CI pipeline.

3.1 Secrets Management

We used GitHub Secrets to store our sensitive information such as Azure credentials, DockerHub token and username, as well as SonarQube token and URL. Using GitHub Secrets ensures that sensitive information is handled securely and efficiently, reducing the risk of exposure. Because secrets are encrypted and only accessible by workflows running in the repository. And can be easily referenced in our workflows without exposing them in the code. The centralized management makes it easier to update or rotate them as needed.

3.2 SAST

We used SonarQube for Static Application Security Testing (SAST) to perform a thorough analysis of our application's source code. This included checking for security

TM and © 2020 - 2024, All Rights Reserved

vulnerabilities, code quality issues, and adherence to coding standards. SonarQube also provided compliant solutions for the issues.

We created a SonarQube instance using the Docker version of SonarQube and Azure Container Apps. Along with that, we also created SQL server, SQL database using azure CLI, and configured Firewall rule that allows access to the SQL server from all Azure.

3.3 DAST

For DAST we used OWASP ZAP to test the application in its running state, simulating real-world attacks to identify vulnerabilities that could be exploited by malicious actors. OWASP ZAP generate detailed reports listing the identified vulnerabilities, their severity level, and recommended fixes. Configuration for Dynamic Application Security Testing (DAST) is more complex than SAST. To execute DAST, we need to have a running application, which means that we must build and deploy the application before we can run the tests.

By integrating SonarQube and OWASP ZAP into our CI pipeline, we ensured that security checks were automated and continuously applied throughout the development lifecycle. This proactive approach helped us identify and address security vulnerabilities early, reducing the risk of security breaches in production.

4. Implementation

4.1 CI pipeline

```
name: Python test and build

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.x'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Test with pytest
        run: |
          pip install pytest pytest-cov
          pytest api_tests.py --doctest-modules --junitxml=junit/test-results.xml --cov=com
--cov-report=xml --cov-report=html
      - name: Archive code coverage results
```

TM and © 2020 - 2024, All Rights Reserved

```
        uses: actions/upload-artifact@v4
        id: artifact-upload-step
        with:
          name: test-results
          path: junit/test-results.xml

  sonarqube:
    needs: test
    if: true
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
        # Disabling shallow clones is recommended for improving the relevancy of
reporting
          fetch-depth: 0
      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_URL }}
      - name: Download SonarQube report
        run: |
          sleep 30
          URL="${{ secrets.SONAR_URL
}}/api/qualitygates/project_status?p=1&projectKey=devsecops-app&branch=main"
            echo "Report URL $URL"
            curl -u ${{ secrets.SONAR_TOKEN }}: "$URL" -o sonarqube-report.json
      - name: Upload SonarQube report
        id: artifact-sonarqube
        uses: actions/upload-artifact@v4
        with:
          name: sonarqube-report
          path: sonarqube-report.json

  dast:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          push: true
```

```
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:dast-test
      - uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
      - name: Deploy DAST Container App
        uses: azure/container-apps-deploy-action@v1
        with:
          imageToDeploy: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:dast-test
          containerAppName: devops-test-app
          containerAppEnvironment: devops-app-env
          resourceGroup: devsecops-lab
          location: australiaeast
          targetPort: 5000
      - name: Get deployed DAST app url
        run: |
          export FQDN=$(az containerapp show -n devops-test-app -g devsecops-lab
--query properties.configuration.ingress.fqdn -o tsv)
          echo "TEST_APP_URL=https://$FQDN" >> $GITHUB_ENV
          echo "TEST_APP_URL=https://$FQDN"
          curl https://$FQDN/ui
          sed -i "s,http://localhost:5000,https://$FQDN,g"
$GITHUB_WORKSPACE/openapi_specs/openapi3.yml
      - name: ZAP Scan
        uses: zaproxy/action-api-scan@v0.6.0
        with:
            token: ${{ secrets.GITHUB_TOKEN }}
            docker_name: 'ghcr.io/zaproxy/zaproxy:stable'
            format: openapi
            target: /zap/wrk/openapi_specs/openapi3.yml
            rules_file_name: .zap/rules.tsv
            cmd_options: '-a -c /zap/wrk/.zap/rules.tsv'
            fail_action: false
            allow_issue_writing: false
      - name: Upload ZAP report
        id: artifact-zap
        uses: actions/upload-artifact@v4
        with:
              name: zap-report
              path: report_json.json

  image-scan:
    needs: test
    if: true
    runs-on: ubuntu-latest
    steps:
        - name: Scan image
          uses: anchore/scan-action@v3
          id: image-scan
          with:
            image: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:latest
            fail-build: false
            output-format: json
        - name: Upload Image report
```

TM and © 2020 - 2024, All Rights Reserved

```
        id: upload-image-report
        uses: actions/upload-artifact@v4
        with:
          name: image-report
          path: ${{ steps.image-scan.outputs.json }}

  security_gate:
    needs: [sonarqube, dast, image-scan]
    if: false
    runs-on: ubuntu-latest
    steps:
    - name: Download sonarqube-report
      uses: actions/download-artifact@v4
      with:
        name: sonarqube-report
        path: ${{github.workspace}}
    - name: Download zap-report
      uses: actions/download-artifact@v4
      with:
        name: zap-report
        path: ${{github.workspace}}
    - name: Download image-report
      uses: actions/download-artifact@v4
      with:
        name: image-report
        path: ${{github.workspace}}
    - name: Read reports
      run: |
        IMAGE_RESULT_HIGH=$(cat results.json | jq '.matches[] |
select(.vulnerability.severity=="High") | .vulnerability.id' | wc -l)
        if [ $IMAGE_RESULT_HIGH -gt 0 ]; then
          echo "Image scan has high severity vulnerabilities"
          exit 1
        else
          echo "Image scan passed"
        fi

        SONAR_RESULT=$(cat sonarqube-report.json | jq -r '.projectStatus.status')
        if [ $SONAR_RESULT != "OK" ]; then
          echo "Sonar scan has issues"
          exit 1
        else
          echo "Sonar scan passed"
        fi

        ZAP_RESULT_HIGH=$(cat report_json.json | jq -r '.site[].alerts[] | select(.riskdesc
| startswith("High")) | .name' | wc -l)
        if [ $ZAP_RESULT_HIGH -gt 0 ]; then
          echo "Zap scan has high severity vulnerabilities"
          exit 1
        else
          echo "Zap scan passed"
        fi
```

TM and © 2020 - 2024, All Rights Reserved

```
        echo "All security gates passed"

  build:
    needs: [sonarqube, dast, image-scan]
    if: true
    permissions:
      id-token: write
    runs-on: ubuntu-latest
    steps:
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          sbom: true
          provenance: true
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:latest
      - name: Install Cosign
        uses: sigstore/cosign-installer@v3.4.0
      - name: Sign the images with GitHub OIDC Token
        env:
          IMAGE: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:latest
        run: |
            cosign sign --yes ${IMAGE}

  submit_reports:
    needs: build
    if: true
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
      - name: Azure CLI script
        id: token-gen
        uses: azure/CLI@v1
        with:
          azcliversion: 2.63.0
          inlineScript: |
            ACCESS_TOKEN=$(az account get-access-token | jq -r .accessToken)
            echo "azureAccessToken=$ACCESS_TOKEN" >> "$GITHUB_OUTPUT"
      - uses: actions/download-artifact@v4
        with:
```

TM and © 2020 - 2024, All Rights Reserved

```
        name: sonarqube-report
        path: ${{github.workspace}}
    - uses: actions/download-artifact@v4
      with:
        name: zap-report
        path: ${{github.workspace}}
    - uses: actions/download-artifact@v4
      with:
        name: image-report
        path: ${{github.workspace}}
    - name: Upload reports
      env:
        DOCKERHUB_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
        AZURE_ACCESS_TOKEN: ${{ steps.token-gen.outputs.azureAccessToken }}
        IMAGE: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs
      run: |
        WEEK1_CONTAINER=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN" -d "$DOCKERHUB_USERNAME/devops-labs"
https://devsecops-ver.azurewebsites.net/api/week1-container)
        echo "Week1 container upload: $WEEK1_CONTAINER"

        SONAR_RESPONSE=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN"  --data-binary
@$GITHUB_WORKSPACE/sonarqube-report.json
https://devsecops-ver.azurewebsites.net/api/week3-sonar-report)
        echo "Sonar upload: $SONAR_RESPONSE"
        ZAP_RESPONSE=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN"  --data-binary
@$GITHUB_WORKSPACE/report_json.json
https://devsecops-ver.azurewebsites.net/api/week3-zap-report)
        echo "Zap upload: $ZAP_RESPONSE"

        IMAGE_RESPONSE=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN" --data-binary @$GITHUB_WORKSPACE/results.json
https://devsecops-ver.azurewebsites.net/api/week4-image-report)
        echo "Image report upload: $IMAGE_RESPONSE"

        WEEK5_RESPONSE=$(curl -X POST  -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN"  -d "$IMAGE"
https://devsecops-ver.azurewebsites.net/api/week5-report)
        echo "Week5 report upload: $WEEK5_RESPONSE"

        MODEL_RESPONSE=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN" --data-binary
@$GITHUB_WORKSPACE/threat-model/ci-pipeline.json
https://devsecops-ver.azurewebsites.net/api/week6-report)
        echo "Model report upload: $MODEL_RESPONSE"
```

4.2 CD pipeline

```
name: Deploy
```

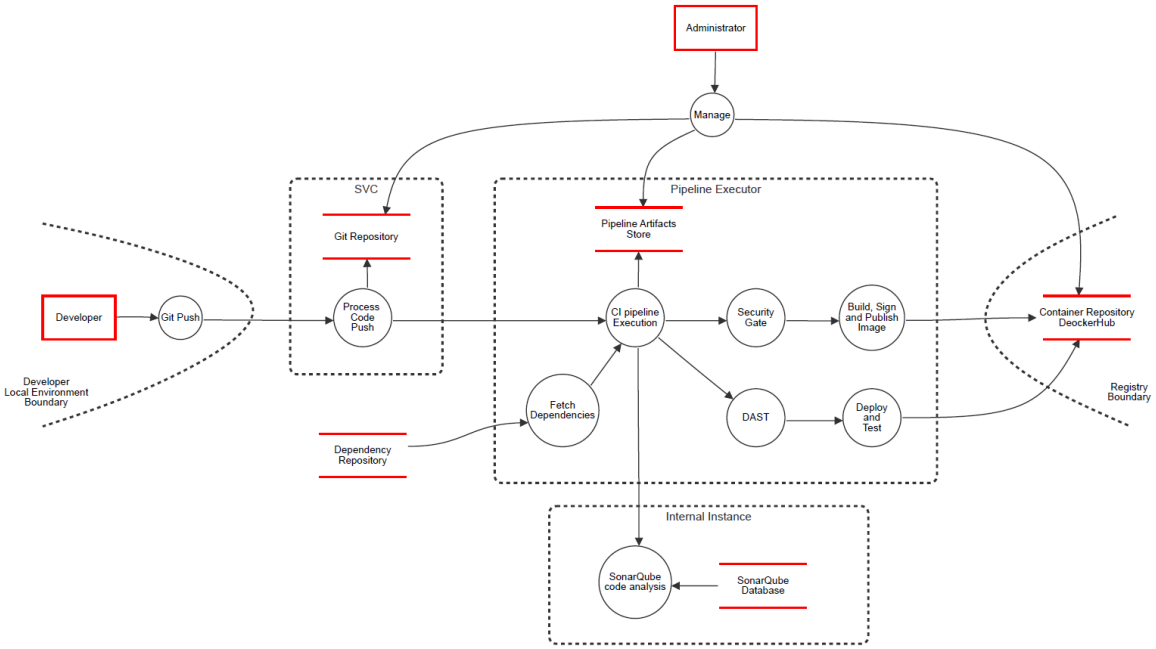
TM and © 2020 - 2024, All Rights Reserved

```
on: workflow_dispatch

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
      - name: Deploy Container App
        uses: azure/container-apps-deploy-action@v1
        with:
          imageToDeploy: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:latest
          containerAppName: devops-app
          containerAppEnvironment: devops-app-env
          resourceGroup: devsecops-lab
          location: australiaeast
          targetPort: 5000
      - name: Install Cosign
        uses: sigstore/cosign-installer@v3.4.0
      - name: Verify image signature
        env:
          IMAGE: ${{ secrets.DOCKERHUB_USERNAME }}/devops-labs:latest
          GITHUB_ISSUER: https://token.actions.githubusercontent.com
          CERT_IDENTITY_REGEXP:
https://github.com/.*/IATD-DevSecOps/.github/workflows/ci-pipeline.yml.*
        run: |
          cosign verify $IMAGE  --certificate-identity-regexp $CERT_IDENTITY_REGEXP
 --certificate-oidc-issuer $GITHUB_ISSUER

  submit_results:
    needs: [deploy]
    if: true
    runs-on: ubuntu-latest
    steps:
      - uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
      - name: Azure CLI script
        id: token-gen
        uses: azure/CLI@v1
        with:
          azcliversion: 2.63.0
          inlineScript: |
            ACCESS_TOKEN=$(az account get-access-token | jq -r .accessToken)
            echo "azureAccessToken=$ACCESS_TOKEN" >> "$GITHUB_OUTPUT"
      - name: Upload reports
        env:
          DOCKERHUB_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
          AZURE_ACCESS_TOKEN: ${{ steps.token-gen.outputs.azureAccessToken }}
        run: |
```

```
        export FQDN=$(az containerapp show -n devops-app -g devsecops-lab --query
properties.configuration.ingress.fqdn -o tsv)
        echo "FQDN: $FQDN"
        WEEK2_SUBMIT=$(curl -X POST -H "Authorization: Bearer
$AZURE_ACCESS_TOKEN" -d "https://$FQDN"
https://devsecops-ver.azurewebsites.net/api/week2-submit)
        echo "Week2 submit: $WEEK2_SUBMIT"
```

5. Threat Analysis

We used OWASP Threat Dragon to create threat analysis. This tool helps in identifying and mitigating potential security threats in our pipeline. The diagram below highlights potential security risks to our CI/CD pipeline.



6. Notes

There are multiple areas to put in consideration while securing a CI/CD pipeline. These include preventing unauthorized access, strengthening authentication, ensuring code integrity to avoid tampering, manage dependencies to prevent vulnerabilities, comprehensive security scanning, and data protection through encryptions. Finally, implementing detailed monitoring and logging to help detect anomalies, ensuring security and resilience of CI/CD process.

7. Summary

To create a secure CI/CD pipeline, we integrated several tools and platforms. We used GitHub as a source code management, Docker Hub for container management, VS Code

TM and © 2020 - 2024, All Rights Reserved

for development, and Azure Cloud to deployment. The pipeline operates within GitHub repository, ensuring smooth integration with Docker Hub and Azure. Security was priority, sensitive information such as credentials and tokens securely managed by GitHub Secrets. We incorporated SAST using SonarQube to help identify vulnerabilities and code quality issues in the source code. We also implemented DAST using OWASP ZAP to test the running application for potential attacks. Additionally, we used OWASP Threat Dragon for threat analysis to help us identify and mitigate potential security threats in our pipeline. This setup allowed us to automate and continuously apply security checks throughout the development lifecycle and reducing the risk of security breaches.

8. Version History

Date	Version	Author	Reviewed by	Approved by	Change History
06-09-2024	1	Ilmi Bilquish Setia	Chayan Dashora IATD Educator	Chayan Dashora IATD Educator	Document created